HabotConnect Hiring Project

Polished Deployment & Automation Blueprint

Junior Cloud & DevOps Engineer (GCP / Django / React)

Candidate: Mayank

Date: August 2026 | Restore staging integrity with IaC, Fail-Closed gates, and schema enforcement

Why This Project Exists

The Staging Incident

- A junior developer bypassed security guidelines
- Unencrypted API credentials left in raw application code
- Database schema mismatch broke downstream analytics
- Human memory alone is not enough to prevent recurrence

Our Response Philosophy

- Build Poka-Yoke (mistake-proofing) systems
- Automate Golden Rules instead of hoping people remember
- Fail-Closed: bad commits stop immediately
- Produce objective evidence: Terraform, CI gates, serializers

Assessment Areas Covered

01

Infrastructure as Code

Provision GCP Storage and BigQuery securely with Terraform

02

Fail-Closed CI Gates

Linter, formatter, and secret scans that halt bad builds

03

Schema & DCYN Validation

Binary Yes/No logic plus exact Django field limits

04

Identity & Access Control

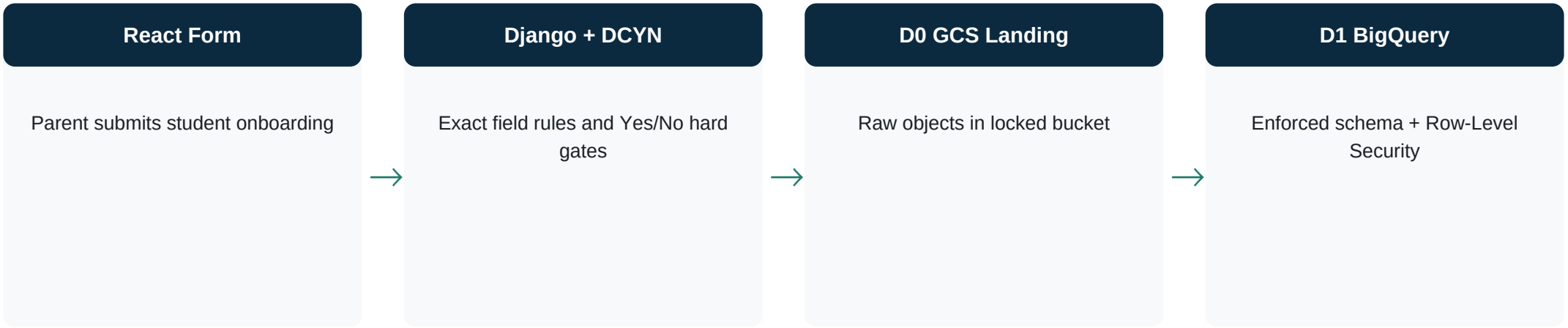
Least Privilege IAM and Row-Level Security

05

Technical Rigor

Full documentation, no placeholders, clear structure

Architecture Overview



CI/CD Fail-Closed Gate sits in front of deploy

Pull Request → Format/Lint → Secret Scan → Schema Validation → Only then Staging Deploy is allowed

Task 1 — D0 Raw Landing Bucket (GCS)

- Uniform bucket-level access enforced (no legacy object ACLs)
- Public access prevention enforced — bucket cannot be made public
- Versioning enabled + soft-delete retention for recovery
- Lifecycle rules move older objects to Nearline, then delete
- Ingest writer gets storage.objectCreator only — not objectAdmin
- Optional Customer-Managed Encryption Key support via variable

Task 1 — D1 Staged/Enforced BigQuery

- Dataset and table for student_onboarding_events
- Schema enforced to match Django serializer and schema-mapping.csv
- Partitioned by ingested_at (day) for cost-efficient queries
- Clustered by learning_support_region and requires_learning_support_assistant
- Deletion protection enabled to prevent accidental table drops
- Prevents silent schema drift into analytics sinks

Task 1 — Identity & Row-Level Security

Least Privilege Identities

- Separate ingest-writer service account
- Separate analytics-reader service account
- Writer: create objects / edit D1 rows
- Reader: query under RLS + jobUser only
- Never assign Owner / Editor / Storage Admin in the data path

Row-Level Security Policies

- Region filter: learning_support_region = authorized region
- Consent filter: consent_to_analytics_export = TRUE
- Readers cannot see other regions' rows
- Readers cannot see non-consented analytics rows
- Uses Google provider 6.x row access policy resource

Task 2 — Poka-Yoke Build Gate Design

Format	Lint	Terraform	Secrets	Schema
Ruff format --check	Ruff check	fmt + validate	Custom scan + Gitleaks	Serializer pass/fail tests

Fail-Closed rule

Any finding → fail job → halt pipeline → quarantine when secrets are detected. There is no warn-and-continue path for security findings.

Fail-Closed Demonstration

Insecure Temporary File

- Script: demonstrate-fail-closed.sh
- Creates temp file with API_KEY = "..."
- Custom secret scanner finds the pattern
- Exit code = non-zero (FAIL)
- Build would be halted and quarantined

Secure Sample

- examples/secure-sample.py
- Loads HABOT_API_KEY from environment
- No hardcoded credentials in source
- Scanner finds zero issues
- Exit code = 0 (PASS)

Quarantine Behavior

- On secret detection, quarantine-build.sh writes artifacts/quarantine/quarantine-<commit>.txt
- Marker records commit SHA, gate name, Coordinated Universal Time timestamp, and action: Do not deploy
- GitHub Actions uploads quarantine artifacts on failure for investigation
- Deploy job uses needs: [all gates] and if: success() — never runs after a failed gate
- Result: insecure commits cannot reach staging through hope or override

Task 3 — Decide Yes or No (DCYN) Library

Hard Gates (Must Be Yes)

- Student age eligible (5 to 18 years)
- Consent to data processing = true
- Weekly support hours between 1 and 40
- Learning support region recognized
- Schema version supported (1.0.0)
- If any hard gate is No → reject payload

Informational Decisions

- Has diagnosed learning difficulty
- Requires Learning Support Assistant
- Analytics export consent (also used by BigQuery RLS)
- Binary only — no maybe / soft pass
- Eliminates human judgment at the API boundary

Task 3 — Django REST Framework Serializer

- Exact validation limits: name length 2–120, email max 254, hours 1–40
- Choice fields for learning_support_region and payload_schema_version
- Boolean fields must be strict true/false — not "maybe" or strings
- validate() runs Decide Yes or No matrix and rejects failed hard gates
- CI proves: sample_payload.json passes; sample_payload_invalid.json fails

Schema Mapping Contract

- Single source of truth: docs/schema-mapping.csv
- Maps JSON fields → Django serializer → BigQuery columns → DCYN gates
- Import into Google Sheets / Excel and enable Wrap Text for full visibility
- Use Full Forms Only — no slang, abbreviations, or placeholders
- Keeps Pub/Sub streaming sinks aligned with transactional schema

Evidence & Repository Map

task1-terraform/
GCS D0, BigQuery D1, IAM, Row-Level Security

task2-cicd/
Scripts for secret scan, quarantine, Fail-Closed demo

.github/workflows/
poka-yoke-build-gate.yml — live CI gate

task3-dcyn/
DCYN library, Django model, serializer, sample payloads

docs/
Architecture, schema mapping, presentation materials

examples/
Secure vs intentional insecure credential patterns

Thank You — Ready for Panel Questions

Prepared to discuss:

- Least Privilege Identity and Access Management design
- Fail-Closed Poka-Yoke continuous integration behavior
- Schema enforcement from Django to BigQuery

Candidate: Mayank

Repository: https://github.com/maynk1013/habotconnect | Add your email and phone before submission